

Fundamentals

Planning Your Approach ·||·

How to decompose the problem and form a plan before you ever open the AI chat.

After you've oriented yourself in the codebase, the next thing to do is form a plan, so you're the one making architectural decisions rather than passively accepting whatever the AI generates. How you plan, how much detail you need, and whether you involve AI in the planning itself all depend on the format and the interviewer. This article walks through the process.

Two approaches to planning

There are two main ways to build your plan, and which one you use depends on the interviewer.

The first is to ask the AI itself for a plan, using something like plan mode in Cursor or Claude, and then carefully vet and revise that plan before executing on it. This is faster and works well when the interviewer is comfortable with you leveraging AI for design thinking. You're still in control because you're evaluating every part of the plan, pushing back where it's wrong, and shaping it into something you fully understand and can defend.

The second is to sketch out the class design and architecture yourself, more like a traditional low-level design exercise, and only bring in AI for implementation. Some interviewers specifically want to see *you* do the design work and will view AI-generated plans as offloading too much responsibility.

This varies by company and even by individual interviewer, so ask the interviewer directly before you start. If they're open to it, the AI-assisted plan is usually the stronger move because you can iterate on it faster. Either way, the rest of this article applies. Whether you built the plan yourself or vetted an AI-generated one, you need to decompose the problem, communicate your plan, use it to guide your prompts, and know when to revise it.

Decompose the problem

Most AI-enabled interview problems have multiple phases. At Meta, the typical structure is fix a bug, implement a feature, then optimize for scale. At Rippling, you build features incrementally with the interviewer providing test cases along the way. Whatever the format, your first job is to break the problem into discrete steps and figure out the order.

For open-ended interviews like Shopify where you're building from scratch, decomposition is especially critical because there's no existing code to guide you. Start by modeling your entities before writing any logic. Defining your data structures first forces you to think through the system before you touch the AI, and it's one of the most effective ways to stay in control. From there, follow a clear ordering:

- Data models before the logic that uses them
- Core functionality before edge cases
- Bug fixes before new features that depend on the broken code working

Without this structure, AI will generate a sprawling mess of code that's hard to refactor when requirements get more specific in later phases. Under time pressure, candidates routinely skip ahead to the interesting part and then have to backtrack.

For structured interviews the decomposition is usually more obvious since the problem statement often spells out the phases. But even then, within each phase, think about how much thinking a step requires versus how much is just implementation volume. The parts that are straightforward boilerplate are great candidates for AI to handle with minimal direction. The parts that require careful reasoning about edge cases, data structures, or algorithm selection are the ones where you need to do the thinking yourself and then direct the AI to implement your specific approach.



A good decomposition usually has 3-5 steps for a 45-60 minute interview. If you've got more than that, you're probably over-splitting. If you've only got one or two, you're probably thinking too broadly and should break it down further.

State your plan out loud

Once you have a plan, say it to the interviewer before you touch the AI. "I'm going to start by fixing the parser bug, then implement the search feature using BFS, then optimize it with a

priority queue if we need to handle weighted edges." That takes ten seconds and it does several important things at once.

First, it shows the interviewer you're thinking strategically. Second, it gives them a window to redirect you if you're heading somewhere unproductive. Third, it creates a shared understanding of what you're trying to accomplish, which makes everything you do afterward easier to follow. If the interviewer knows you're working on step two of your plan, they can contextualize your prompts and your decisions without you having to explain every little thing in real time.

Guide AI with your strategy

Your plan should directly shape how you prompt. Break your prompts along the same lines as your decomposition, one step at a time, with specific direction about your chosen approach.

There's a meaningful difference between "solve this problem" and "implement a BFS traversal of the maze, starting from the `start` position in the `Grid` class, returning the shortest path as a list of coordinates." The first prompt gives the AI full control over the approach. The second tells the AI exactly what to build, using the vocabulary of the codebase you've already oriented yourself in. The second gets better results, and it shows the interviewer that you're directing the work.



It's possible to be too specific. If you're dictating every variable name, line structure, and implementation detail, you're not really directing the AI. You're writing the code through it. The goal is to give the AI enough direction to implement your approach correctly while leaving it room to handle the syntax and structure. That's also what gives the interviewer a chance to evaluate you, not just watch Claude type.

Be specific about data structures, algorithm choices, and how your code should interact with what already exists. If you know the codebase has a `Graph` class with an `adjacencyList` property, reference it in your prompt. The AI will produce code that fits into the existing architecture instead of inventing its own parallel structure. This matters because you need to be able to explain everything in the codebase. If the AI drifts into reinventing your architecture,

you lose contact with what's happening. An interviewer asking "what does this class do?" will expose that immediately.

When the AI comes back with something, check whether it actually implemented what you asked for. AI models will sometimes swap your chosen algorithm for a different one, or add unnecessary abstractions you didn't ask for. Catch these deviations early and course-correct before building on top of them.



One pattern that gets candidates in trouble is prompting the AI with the raw problem statement instead of their own plan. The AI will produce *a* solution, but it might not be the *right* solution for the codebase or the interviewer's expectations. Always translate the problem through your own understanding before asking the AI to implement it.

Know when to revise the plan

Plans should change when you learn something new. Maybe you discover a performance constraint you didn't anticipate. Maybe the data model is more complex than it looked during orientation. Maybe the interviewer drops a hint that changes the problem scope. All of these are legitimate reasons to revise your approach, and doing so shows adaptability.

Plans should not change just because the AI suggests something different. If you abandon your approach every time the AI offers an alternative, you're no longer driving. The test is whether you can articulate *why* the new approach is better for the problem at hand. If the AI suggests a different data structure and you can explain why it's a better fit, that's a good pivot. If the AI suggests refactoring the entire class hierarchy and you can't explain why that would help, that's scope creep. Trust your plan until you have a concrete reason not to.



Watch for the AI spiral. When a model doesn't know the answer, it often starts proposing rewrites, restructuring your architecture, or suggesting you start over from scratch. This can feel like momentum but it's actually the AI flailing. Interviewers want to see you recognize that pattern and interrupt it. Step back, reassess, and redirect rather than following the AI down a rabbit hole.

When you do pivot, say so explicitly. "I realized the brute force approach won't scale for the larger inputs in phase three, so I'm switching to a heap-based solution." This narration turns what could look like confusion into a demonstration of adaptive problem-solving, which is exactly how senior engineers work on real projects.